Eco-Guide: Technical Report

Rui Silva - 1251443
Gonçalo Matos – 1241270

Abstract. This Report documents the development of the android app Eco Guide. Throughout the report, the whole process of making the application will be clearly evidenced , as well as what came out of the work put into it.

Keywords: API, android, java, app, Eco Guide, Recycling, Gamification, GPS

1 Context

As part of the software and mobile development course, we were asked to make android project using java programming language. Our project, named Eco Guide, consist of promoting and managing urban recycling through an API. There, the user can search for nearby Eco points based on geographic location and categories (such as glass, plastic/metal, paper and batteries). Users can record their recycling deposits by specifying the material type and quantity, which automatically computes and updates their ecological points. Additionally, the application features a gamification and ranking system where users unlock badges and track their environmental impact over time. We implemented a secure registration and login system, where user accounts and associated recycling data are persistently stored. This allows multiple users to maintain individual profiles and personalized recycling histories within the application

2 Analysis

2.1 Domain Model

As illustrated in Figure 1, the application´s domain is centered around the User, who accumulates total ecological points. Through Recycling Records, users register their recycling actions by specifying the quantity of different waste sacks deposited (yellow, blue,

green). Each record stores the points earned from that action and is associated with a physical Eco Point, which possesses its own geographical coordinates.

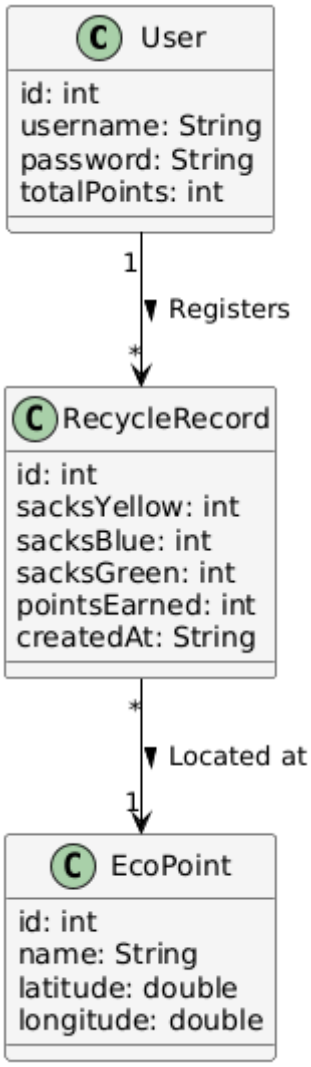


Figure 1 - Domain Model Diagram

2.2 Non-functional Requirements

2.2.1 Usability

- Graphical Android Interface

2.2.2. Reliability

- Robust Exception handling to prevent possible crashes during invalid data entries or duplicate entity creation

2.2.3 Performance

- Must occupy a low memory

- Mustn’t use a lot of phone charge

2.2.4 Supportability

- Android platform

- Minimal android API level support

2.2.5 Design Constraints

- Adherence to the Model-View-Controller (MVC) architectural pattern to ensure clean separation of concerns

2.2.6 Implementation Constraints

- Java Language

2.2.7 Interface Constraints

- Android Graphical User Interface

2.3 Functional Requirements

The Overall Use Case Diagram (Figure 2) illustrates the interactions between the user and Eco Guide system. This model represents the complete and continuous flow of the application, tracking the user´s journey from accessing the platform to their participan in the gamification's dynamics.

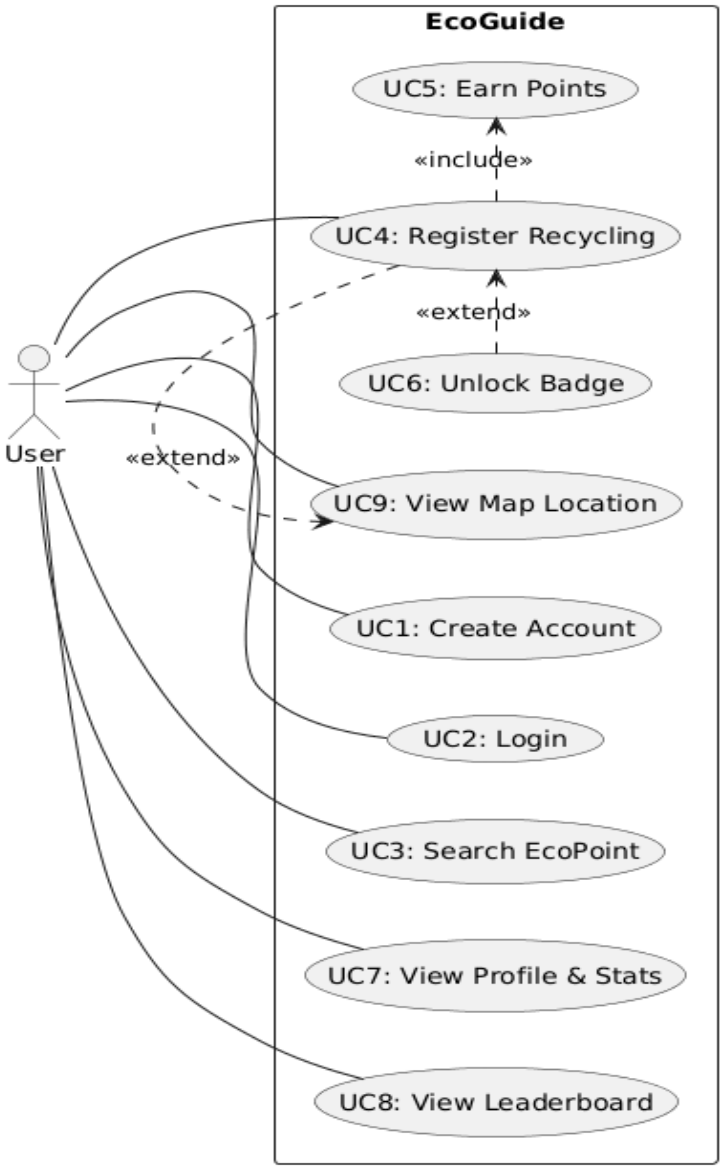


Figure 2 - Use Case Diagram

The application flow begins with identity management (UC1 and UC2), allowing the user to securely access their account. Once authenticated, the user can locate Eco Points through direct search (UC3) or interactive map visualization (UC9). From the map interaction, the user can proceed directly to the application´s core functionally: registering their recycling actions (UC4). To support the gamification, the recycling registration triggers dependent processes are represented by some specific relationships: it obligatory including point allocations (<< include >> UC5) and conditionally unlocks new badges (<< extend >> UC6) if specific goals are met. After all progress being generated during this flow, it can be consulted by the user in their personal profile (UC7) or on the global leaderboard (UC8), promoting competitiveness and continuous application usage.
Figure 3 and Table 1 detail the Create Account User Case flow

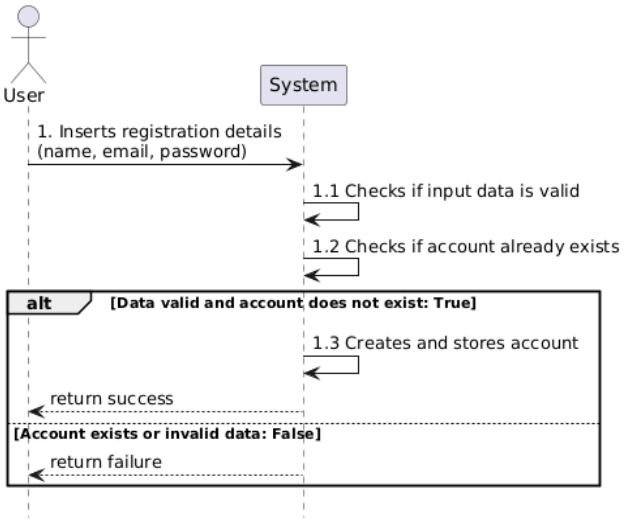


Figure 3 - UC1 Create account

Description	Creates a new user account in the system
Pre-condition	None
Post-condition	The user account is created and stored in the system
Basic Path	1. The user enters registration details 2. The system verified if the entered data is valid 3. The system verified if the account already exists 4. The system creates and stores the new account The system returns success
Alternative Path	• Invalid data/Account exists • Return failure

Table 1 - UCI Specifications

Figure 4 and Table 2 detail Login Account use case flow.

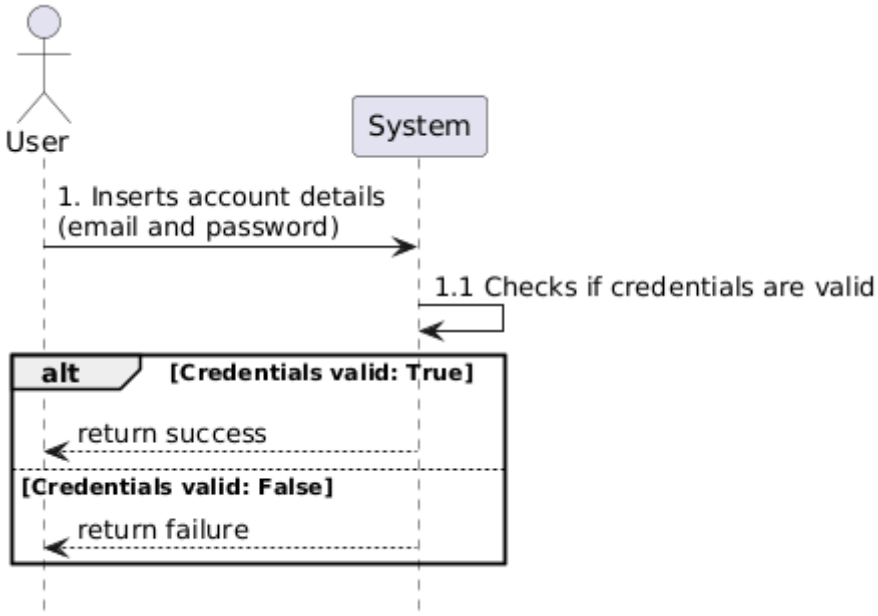


Figure 4 - UC2 Login Account

Description	Authenticates the user based on email or password
Pre-Condition	The account must already exist
Post-Condition	The user accesses the account and session is initiated
Basic Path	1. The user enters credentials (email and password) 2. The system verifies if credentials are valid 3. The system grants access to the accounts. The system returns success
Alternative Path	• **Invalid credentials** The system returns failure

Table 2 - UC2 Specifications

Figure 5 and Table 3 details Search Eco Point use case flow.

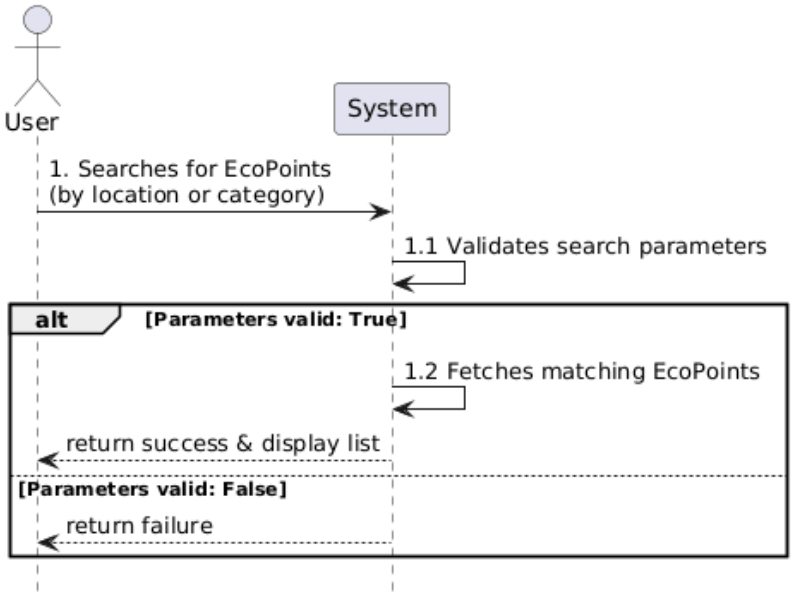


Figure 5 - UC3 Search Eco Point

Description	Enables users to search and view available Eco Points by location or category
Pre-Condition	The user must be logged in
Post-Condition	Matching Eco points are displayed to the user
Basic Path	1. The user inputs search criteria (location or category) 2. The system validates search parameters 3. The system queries and retrieves matching Eco Points 4. The system displays the list of Eco Points The system returns success
Alternative Path	• **Invalid search criteria /No Eco Points found** Returns Failure

Table 3 - UC3 Specification

Figure 6 and Table 4 details Register Recycling use case flow

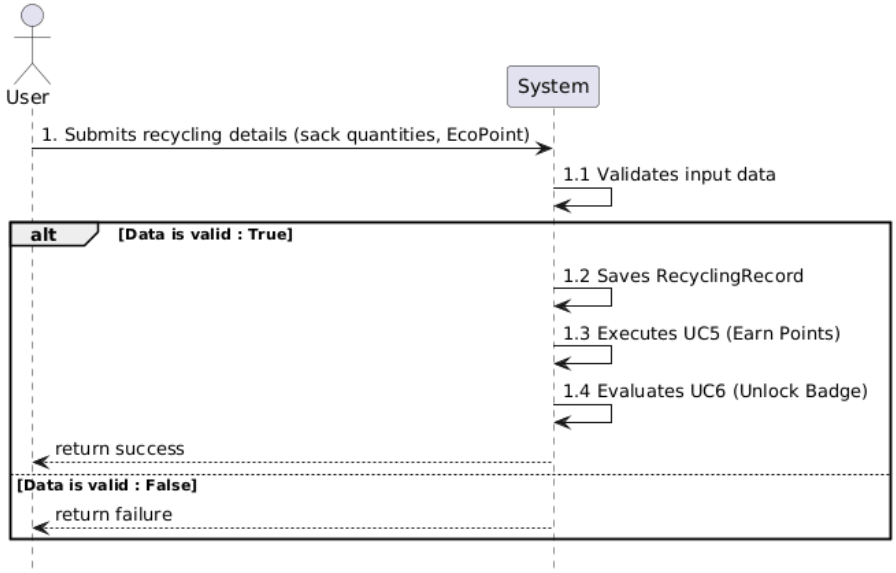


Figure 6 - UC4 Register Recycling

Description	Allows the user to register a recycling action at a specific Eco Point
Pre-Condition	The user must be logged in and the Eco Point must be selected
Post-Condition	A new Recycling Record is created and the Points are updated
Basic Path	1. The user inputs the quantities of different waste sacks. 2. The system validates the data 3. The system saves a Recycling Record 4. The system calculates and adds the points 5. The system returns success
Alternative Path	• **Invalid data** The system returns failure

Table 4 - UC4 Specifications

Figure 7 and Table 5 details Earn Points use case flow

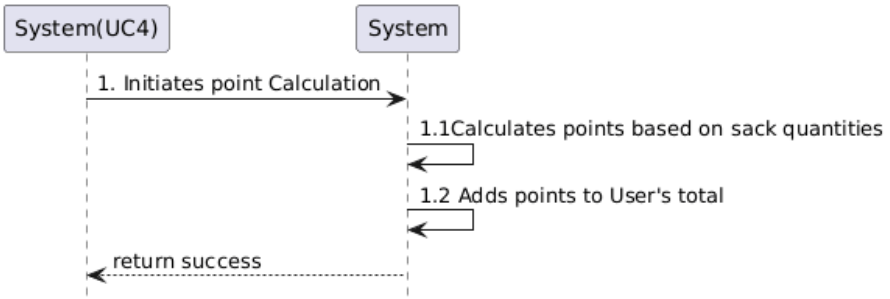


Figure 7 - UC5 Earn Points

Description	Automatically calculates and assigns ecological points after a valid recycling record
Pre-Condition	The register of the recycling Record must be successful executed
Post-Condition	The user´s accumulated ecological points are updated
Basic Path	1. The system retrieves the point multiplier for each sack type 2. The system calculates total earned points 3. The system updates the user´s total score 4. The system returns success
Alternative Path	• **Calculation Error** The system logs an error and returns failure

Table 5 - UC5 Specification

Figure 8 and Table 6 details Unlock Badge use case flow.

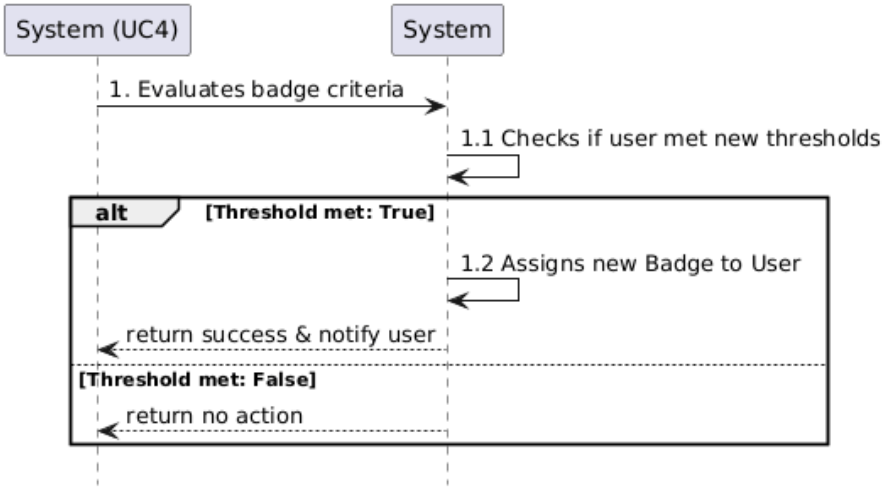


Figure 8 - UC6 Unblock Badge

Description	Conditionally awards a badge if the user reaches specific recycling milestones
Pre-Condition	The register of the recycling Record must be successful executed
Post-Condition	A new badge is associated with the user´s profile
Basic Path	1. The system checks the user´s current points and records against badge threshold 2. The users meet the threshold 3. The system assigns the corresponding Badge 4. The system notifies the user
Alternative Path	• **Threshold not met** The system performs no action

Table 6 - UC6 Specifications

Figure 9 and Table 7 details View Profile & Stats use case flow.

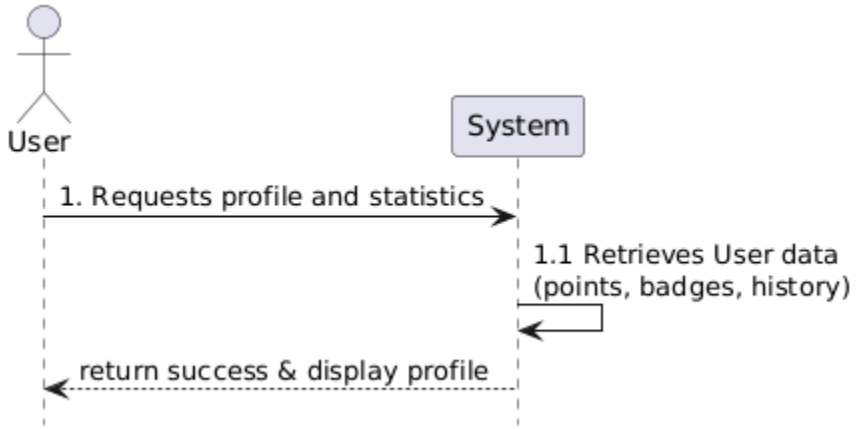


Figure 9 - UC7 View Profile & Status

Description	Conditionally awards a badge if the user reaches specific recycling milestones
Pre-Condition	The register of the recycling Record must be successful executed
Post-Condition	A new badge is associated with the user´s profile
Basic Path	5. The system checks the user´s current points and records against badge threshold 6. The users meet the threshold 7. The system assigns the corresponding Badge 8. The system notifies the user
Alternative Path	• **Threshold not met** The system performs no action

Table 7 - UC7 Specification

Fig 10 and Table 8 details View Leaderboard use case flow.

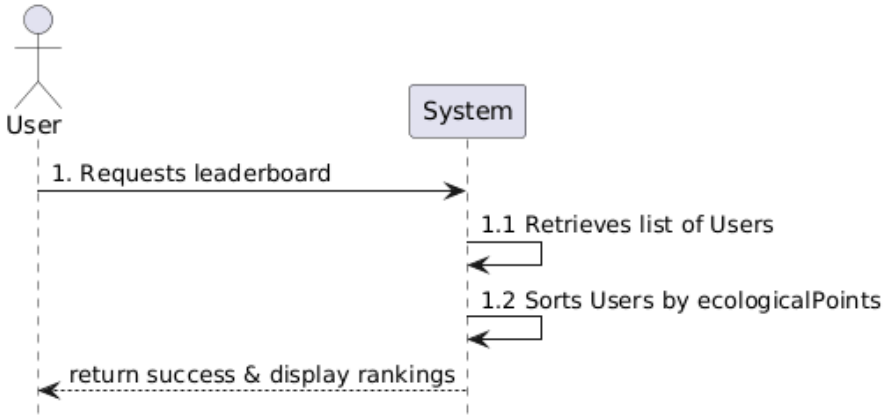


Figure 10 - UC8 View Leader Board

Description	Displays a ranked list of users based on their accumulated ecological points
Pre-Condition	The user must be logged in.
Post-Condition	The ordered leaderboard is presented to the user.
Basic Path	1. The user navigates to the leaderboard screen 2. The system retrieves all users from the database 3. The system sorts the list in descending order based on points 4. The system displays the rankings 5. The system returns success
Alternative Path	• Error fetching data The system returns failure

Table 8 - UC8 Specifications

Fig 11 and Table 9 details View Map Location use case flow.

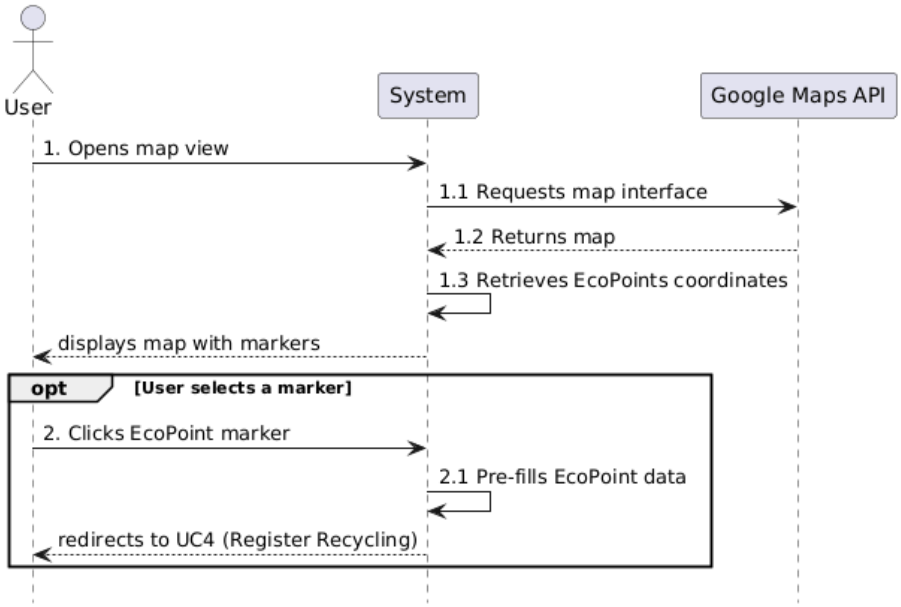


Figure 11 - UC9 View Map Location

Description	Displays Eco Points on a Google Maps Interface and allows selecting an Eco Point to register recycling.
Pre-Condition	The user must be logged in. Device must have network access.
Post-Condition	Map is displayed. User can navigate to UC4 with pre-filled data.
Basic Path	1. The user opens the map section 2. The system requests data from Google Maps API 3. The system loads Eco Points coordinates from the database 4. The system displays markers on the map 5. The user selects a marker, redirecting to UC4
Alternative Path	• Network error/API failure The system displays an error message indicating maps cannot be loaded

Table 9 - UC9 Specifications

3 Design

This section outlines the architectural and visual foundations of the Eco Guide application. It details the physical and logical architectures, demonstrating how the system components interact over the network and within the Model-View-Controller (MVC) pattern. Furthermore, it presents the internal code organization, the system's class diagram and the graphical user interface, illustrating how the structural design supports the application's functional requirements and ensures future maintainability

3.1 Physical Architecture

Physical Architecture describes where the different parts of the system are housed. As shown in **Figure 12**, in Eco Guide project, the application runs on a local Android device and communicates with a remote database.

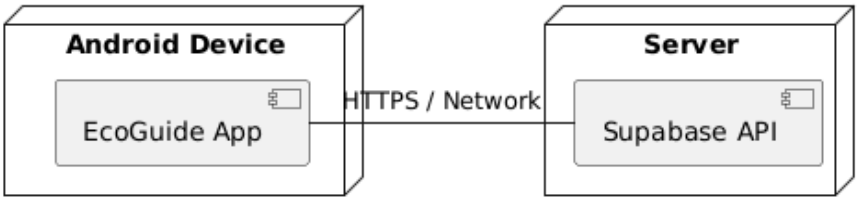


Figure 12 - Physical Architecture

3.2 Logical Architecture

Logical Architecture shows how intern software components interact. As depicted in **Figure 13,** Eco Guide uses MVC pattern, separating the Interface (UI/Activities) of data (Model) and communication with network (Data/API).

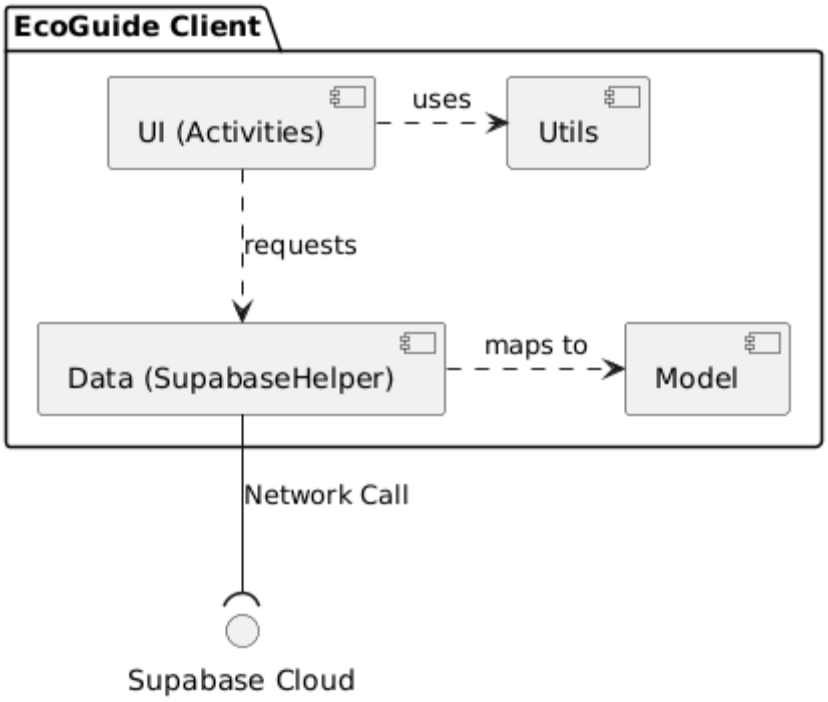


Figure 13 - Logical Architecture

3.3 Implementation

3.3.1 Code Organization

Figure 14, illustrates the internal code organization of the project , mapping the Java packages and XML resources to the actual project folder structure.

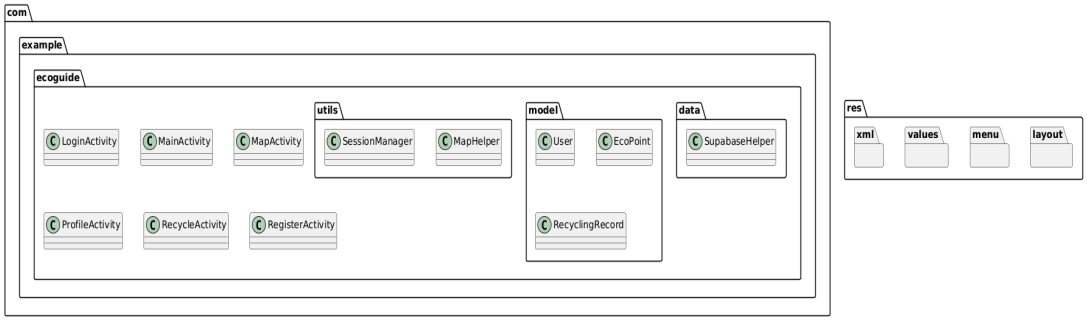


Figure 14 - Code Organization

3.3.2 Class Diagram

This section presents the detailed class diagram of the Eco Guide Application. Unlike the conceptual domain model, this design-level diagram includes the specific attributes, data types, and primary methods for each entity, reflecting the actual Java implementation and the structural relationships within the system, as presented in **Figure 15**.

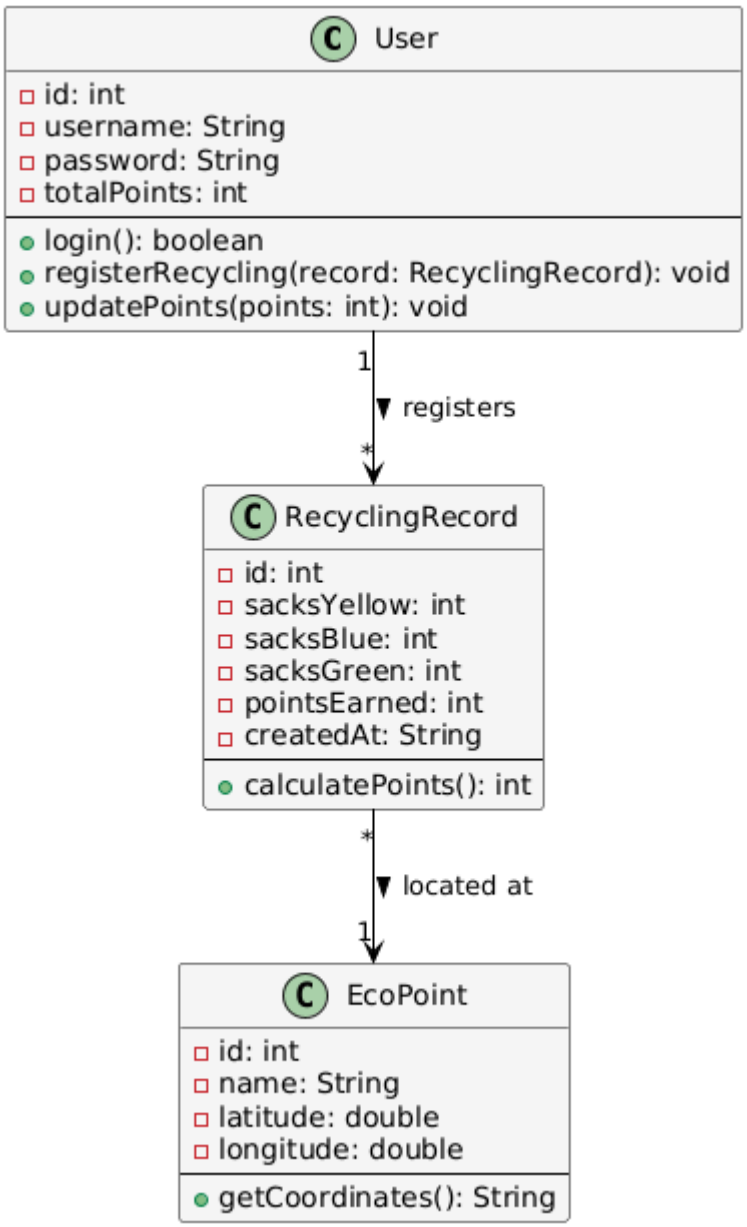


Figure 15 - Eco Guide Class Diagram

3.4 User Interface

This section presents the graphical user Interface (GUI) of the Eco Guide application, illustrating how users interact with the system across different features and screens.

3.4.1 User Authentication (Login & Registration)

As illustrated in **Figure 16** and **Figure 17**, the authentication flow allows users to access their existing accounts or register a new account within the system.

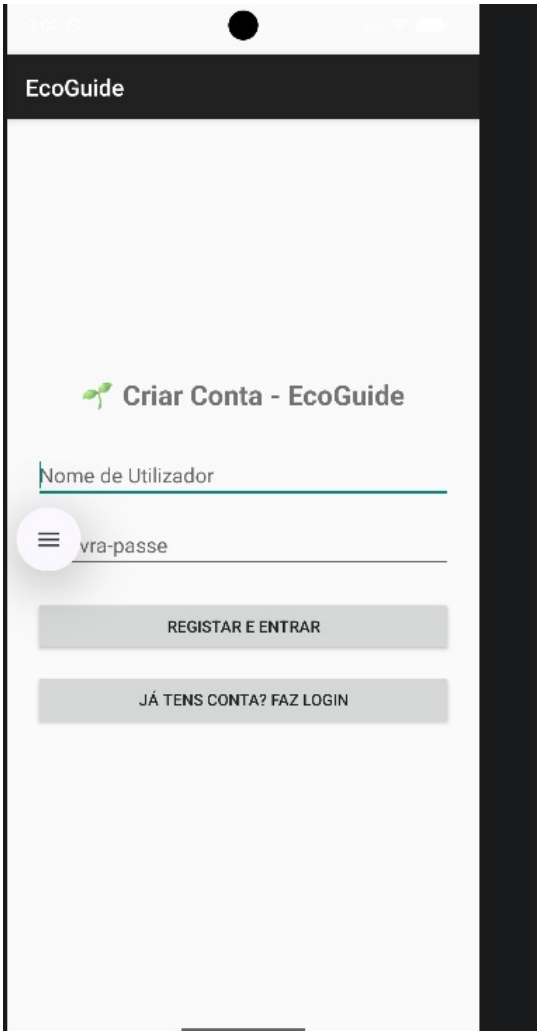


Figure 16 - Registration account screen



Figure 17 - Login account screen

By entering valid credentials (username and password), users can authenticate into the application. If the user does not have an account, clicking the registration button redirects them to the account creation interface, where a new profile is initialized.

3.4.2 Main Dashboard & Profile

Upon successful authentication, the user is presented with the main dashboard shown in the **Figure 18.**

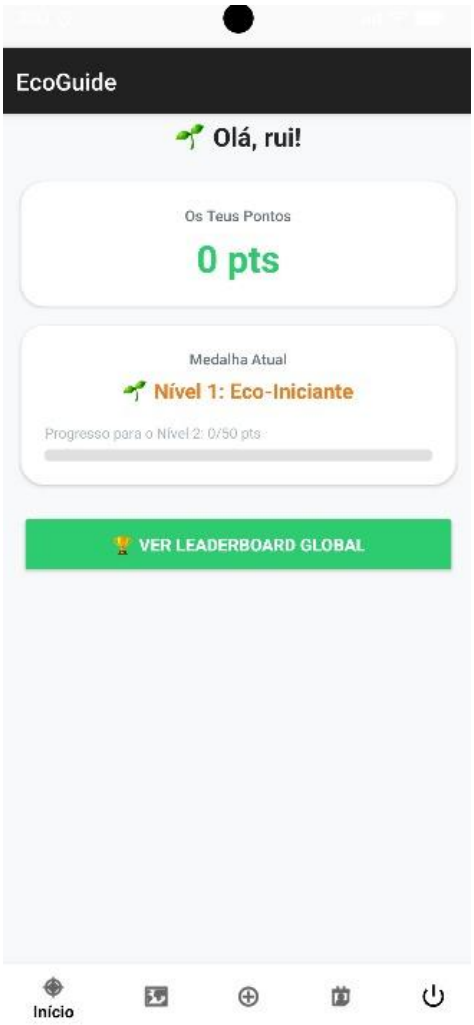


Figure 18 - Main Dashboard & User Profile

This interface displays the user's current ecological points, their active badge level, and a progress bar indicating how many points remaining to unlock the next level.
Additionally, it provides direct navigation to the global leaderboard and includes a bottom navigation bar for quick access to key application features.

3.4.3 Global Leaderboard

Figure 19 displays the global leader boarding ranking users based in their environmental performance.

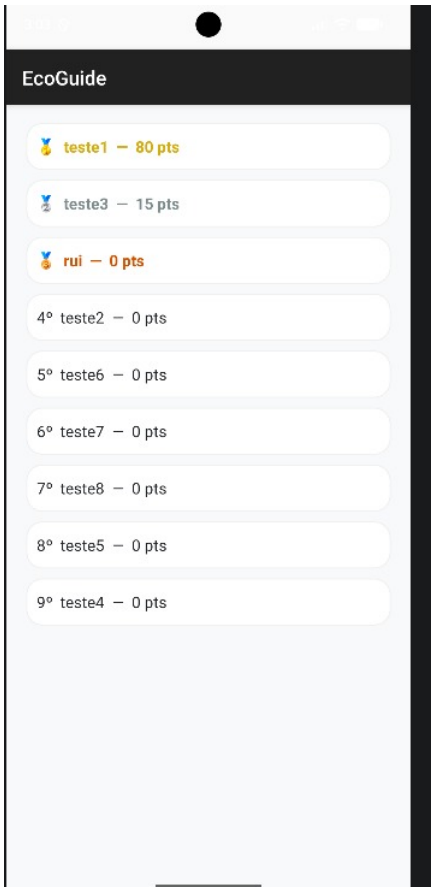


Figure 19 - Global Leaderboard Screen

The leaderboard orders all registered users in descending order according to their accumulated ecological points, awarding distinct visual icons for top positions to foster healthy competition and continuous recycling habits.

3.4.4 Interactive Map Location

The Google Maps integration is depicted in **Figure 20**, enabling users to locate nearby Eco Points visually.



Figure 20 - Interactive Google Maps Interface

Users can view Eco Point locations directly on an interactive map. Clicking the “Eco Ponto Mais Proximo” button automatically identifies and selects the closest recycling spot relative to the user’s current coordinates.

3.4.5 Recycling Action Registration

The **Figure 21** shows the recycling record interface used to submit recycled materials and earn points.

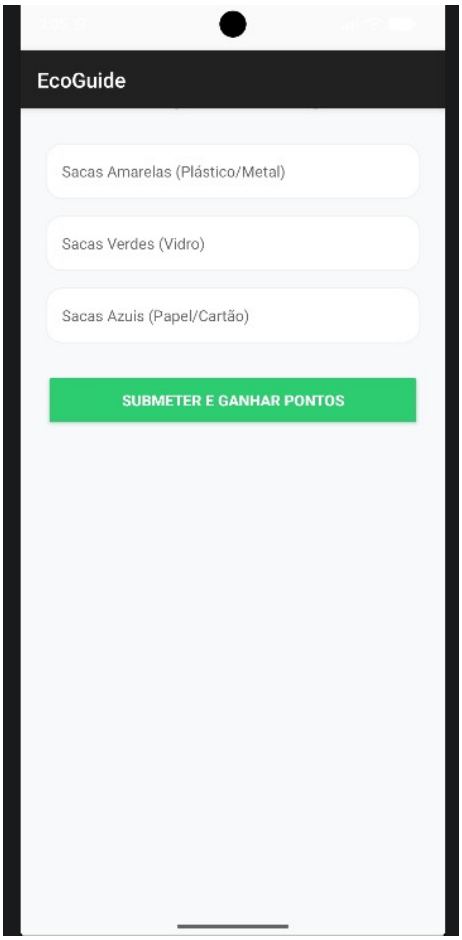


Figure 21 - Recycling Registration Screen

Users can specify the quantity of waste being recycled across different material categories (Yellow/Plastic, Green/Glass, Blue/Paper/Cardboard). Submitting the form validates the input, calculates the earned points, and updates the user's total score.

3.4.6 Badges & Gamification Achievements

Figure 22 details the achievement list available to the user.



Figure 22 - Badges and Achievements Screen

This screen outlines all available badges, displaying unlocked achievements with green checkmarks alongside locked milestones that indicate the point thresholds requires to earn them.

4 Conclusion

Overall, we feel like we accomplished the main goal of this project, as every proposed functionally for the **Eco Guide** application – ranging from user authentication and Google Maps visualization to recycling registration, point calculation, badge unlocking, and leaderboard tracking – is fully working.

During the development process, we faced some challenges with the code structure, particularly when aligning with the ideal Model – View – Controller (MVC) architecture and refining our domain model to adhere strictly to proper software engineering practices. However, we managed to pull through these structural difficulties through iterative refactoring and design adjustments.

Despite the architectural hurdles encountered along the way, we successfully delivered a functional and reliable Android application that we feel confident about, meeting all established project requirements.